JAVA INTERVIEW PREPARATION

CHAPTER 69

GRAPHS AND
GRAPH ALGORITHMS

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

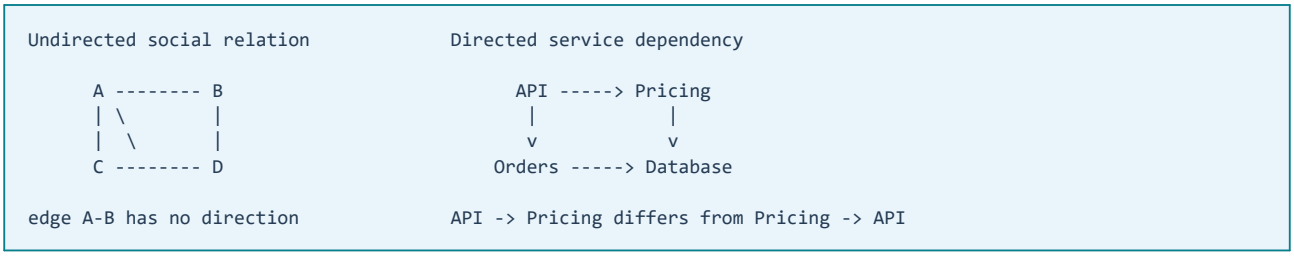
Graphs and Graph Algorithms: Connectivity, Paths, and Dependencies

Senior / Lead Java Developer Reading Chapter

A graph models entities and relationships without requiring a single root or one-parent hierarchy. Vertices may represent services, people, tasks, airports, accounts, states, or database records. Edges may represent calls, friendships, prerequisites, routes, money transfers, or transitions. That generality makes graphs one of the most important data structures for senior engineering interviews and one of the easiest to under-explain.

The central skill is not memorizing a collection of algorithms. It is identifying what an edge means, whether direction and weight matter, which invariant makes a traversal safe, and what scale or mutation model the representation must support. A graph algorithm can be perfectly implemented yet answer the wrong question because the graph was modeled incorrectly.

Graph vocabulary changes the problem



A directed edge has a source and destination. An undirected edge represents a symmetric relation and is normally stored in both adjacency lists. A weighted graph attaches cost, distance, latency, capacity, or risk to an edge. A path is a sequence of connected vertices. A cycle returns to an already visited vertex through one or more edges. A connected component is a maximal connected region in an undirected graph. In directed graphs, weak and strong connectivity are different concepts.

A simple graph has no parallel edges or self-loops; a multigraph may permit both. Do not silently deduplicate routes when multiple edges represent different carriers, prices, or validity windows. The data model must preserve the domain semantics.

Representation determines cost and meaning

An adjacency list stores outgoing neighbors for each vertex.

```
record Edge(int to, long weight) {}

final class Graph {
    private final List<List<Edge>> outgoing;

    Graph(int vertices) {
        outgoing = new ArrayList<>(vertices);
        for (int i = 0; i < vertices; i++) {
            outgoing.add(new ArrayList<>());
        }
    }

    void addDirected(int from, int to, long weight) {
        checkVertex(from);
        checkVertex(to);
        outgoing.get(from).add(new Edge(to, weight));
    }

    List<Edge> outgoingFrom(int vertex) {
        checkVertex(vertex);
        return List.copyOf(outgoing.get(vertex));
    }
}
```

For `V` vertices and `E` directed edges, adjacency-list space is O(V + E). Traversing all vertices and edges is O(V + E). Checking whether one specific edge exists may be proportional to the source's degree unless each neighbor collection is a set or map.

An adjacency matrix uses `matrix[u][v]`. It provides O(1) edge lookup but consumes O(V squared) space, even for sparse graphs. It suits small dense graphs or algorithms that repeatedly inspect every possible pair. A bit matrix can reduce constants for unweighted boolean relationships.

```
Graph: A -> B, A -> C, C -> B

          destination
          A  B  C
source A  0  1  1
       B  0  0  0
       C  0  1  0
```

For business identifiers such as strings or UUIDs, a graph may use `Map<Id, List<Edge<Id>>>`, or map external IDs to dense integer indices. Dense indices reduce memory and enable primitive arrays for visited state and distances. The mapping becomes part of snapshot/version management.

Mutable vertex objects are risky keys if their equality or hash code changes. Prefer immutable identifiers. Exposing mutable adjacency lists also lets callers violate edge rules; return read-only views or encapsulate mutation.

Breadth-first search explores layers

Breadth-first search, or BFS, uses a queue. It completes every vertex at distance `d` before vertices at distance `d + 1`. In an unweighted graph, this proves the first discovered path to a vertex has the minimum number of edges.

```
               A
             /   \
            B     C
           / \     \
          D   E ----- F

BFS from A

distance 0: A
distance 1: B, C
distance 2: D, E, F
```

The memory image is the changing frontier:

```
Step    remove    queue after adding undiscovered neighbors
 0        -       [A]
 1        A       [B, C]
 2        B       [C, D, E]
 3        C       [D, E, F]
 4        D       [E, F]
 5        E       [F]
 6        F       []
```

Mark a vertex when it is enqueued, not when it is removed. Otherwise two parents can enqueue the same vertex repeatedly, increasing work and complicating predecessor assignment.

```java
static int[] shortestUnweighted(List<List<Integer>> graph, int source) {
    int[] distance = new int[graph.size()];
    Arrays.fill(distance, -1);
    Deque<Integer> queue = new ArrayDeque<>();

    distance[source] = 0;
    queue.offerLast(source);

    while (!queue.isEmpty()) {
        int from = queue.pollFirst();
        for (int to : graph.get(from)) {
            if (distance[to] == -1) {
                distance[to] = distance[from] + 1;
                queue.offerLast(to);
            }
        }
    }
    return distance;
}
```

Time is O(V + E) with adjacency lists. The queue can grow to O(V). To reconstruct a path, store `parent[to] = from` at discovery, then walk backward from destination.

Multi-source BFS starts by enqueuing every source with distance zero. It computes distance to the nearest source and appears in contagion, nearest-facility, and grid-expansion problems.

Bidirectional BFS can reduce explored state when one source and one destination are known in a large unweighted graph. Search forward from the source and backward from the destination, expanding the smaller frontier until the visited regions meet. If branching factor is `b` and shortest distance is `d`, two frontiers may explore on the order of `b` raised to `d/2` rather than `b` raised to `d`. The reverse search requires reverse edges for directed graphs, and the meeting logic must still reconstruct one valid path.

```
source frontier ---> ... <--- destination frontier
                        ^
                    first meeting
```

Depth-first search explores one branch

Depth-first search, or DFS, follows one path until it cannot continue, then backtracks. Recursion uses the call stack; an explicit stack avoids call-stack overflow and makes pause/resume state visible.

```
same graph, one possible DFS from A

A -> B -> D
         backtrack to B
         B -> E -> F
                   backtrack
     backtrack to A
     A -> C
```

```java
static void depthFirst(List<List<Integer>> graph, int source) {
    boolean[] visited = new boolean[graph.size()];
    Deque<Integer> stack = new ArrayDeque<>();
    stack.push(source);

    while (!stack.isEmpty()) {
        int vertex = stack.pop();
        if (visited[vertex]) continue;
        visited[vertex] = true;
        visit(vertex);

        List<Integer> neighbors = graph.get(vertex);
        for (int i = neighbors.size() - 1; i >= 0; i--) {
            int neighbor = neighbors.get(i);
            if (!visited[neighbor]) stack.push(neighbor);
        }
    }
}
```

Neighbor order controls one possible traversal order but not reachability. Tests should not require a unique DFS order unless adjacency ordering is part of the contract.

DFS is a foundation for cycle detection, topological sorting, connected components, articulation reasoning, and strongly connected components. Recursive DFS is concise, but an adversarial chain of millions of vertices can overflow the Java stack. An explicit frame may need both vertex and next-neighbor index when postorder behavior matters; merely pushing vertices is not always equivalent to recursive entry/exit timing.

Connected components

For an undirected graph, start BFS or DFS from every unvisited vertex. Each start identifies one connected component.

```
component 1        component 2        isolated component

 A -- B -- C         D -- E                  F
```

```
static int countComponents(List<List<Integer>> graph) {
    boolean[] visited = new boolean[graph.size()];
    int components = 0;

    for (int start = 0; start < graph.size(); start++) {
        if (visited[start]) continue;
        components++;
        Deque<Integer> queue = new ArrayDeque<>();
        visited[start] = true;
        queue.offer(start);
        while (!queue.isEmpty()) {
            int from = queue.poll();
            for (int to : graph.get(from)) {
                if (!visited[to]) {
                    visited[to] = true;
                    queue.offer(to);
                }
            }
        }
    }
    return components;
}
```

If vertices appear only through edges, decide whether absent identifiers count as isolated vertices. Data ingestion must also define duplicate and dangling edges.

Cycle detection differs by direction

For an undirected graph, encountering a visited neighbor is not automatically a cycle because the neighbor may be the parent edge just traversed. Track the parent; a visited neighbor other than the parent proves a cycle. Parallel edges require special handling because two edges between the same vertices may form a length-two cycle in a multigraph.

For a directed graph, use three states:

```
WHITE = not entered
GRAY  = active on current DFS path
BLACK = completely processed

edge to GRAY -> directed back edge -> cycle
edge to BLACK -> already completed, not evidence of a cycle
```

```
static boolean hasDirectedCycle(List<List<Integer>> graph) {
    byte[] state = new byte[graph.size()];
    for (int v = 0; v < graph.size(); v++) {
        if (state[v] == 0 && cycleFrom(v, graph, state)) return true;
    }
    return false;
}

static boolean cycleFrom(int v, List<List<Integer>> graph, byte[] state) {
    state[v] = 1;
    for (int next : graph.get(v)) {
        if (state[next] == 1) return true;
        if (state[next] == 0 && cycleFrom(next, graph, state)) return true;
    }
    state[v] = 2;
    return false;
}
```

Again, replace recursion when graph depth is not safely bounded.

Topological ordering exposes prerequisites

A topological order lists every vertex after all of its prerequisites. It exists only for a directed acyclic graph, or DAG.

```
Compile -> Test -> Package -> Deploy
   \                   ^
    -> Static Analysis -|

valid order: Compile, Test, Static Analysis, Package, Deploy
```

Kahn's algorithm counts indegrees, enqueues all zero-indegree vertices, removes one, and decrements its outgoing neighbors. If fewer than V vertices are emitted, a cycle prevents completion.

```
static List<Integer> topologicalOrder(List<List<Integer>> graph) {
    int[] indegree = new int[graph.size()];
    for (List<Integer> edges : graph) {
        for (int to : edges) indegree[to]++;
    }

    Deque<Integer> ready = new ArrayDeque<>();
    for (int v = 0; v < indegree.length; v++) {
        if (indegree[v] == 0) ready.offer(v);
    }

    List<Integer> order = new ArrayList<>(graph.size());
    while (!ready.isEmpty()) {
        int from = ready.poll();
        order.add(from);
        for (int to : graph.get(from)) {
            if (--indegree[to] == 0) ready.offer(to);
        }
    }
    if (order.size() != graph.size()) {
        throw new IllegalStateException("dependency cycle");
    }
    return order;
}
```

Multiple valid orders may exist. A priority queue can select a deterministic smallest-ready vertex, changing cost to O((V + E) log V). In workflow systems, discovering a cycle should produce a useful cycle path, not only a count mismatch.

Shortest paths depend on edge weights

Algorithm choice follows the weight contract:

```
edge model                                suitable technique

unweighted or every weight equal          BFS
weights are only 0 or 1                    0-1 BFS with a deque
all weights nonnegative                    Dijkstra
negative edges, no reachable negative cycle Bellman-Ford
all pairs, dense/small graph               Floyd-Warshall or repeated search
```

Dijkstra maintains tentative distances and repeatedly finalizes the reachable vertex with the smallest distance. A min-priority queue holds frontier entries.

```
          4
      A ------> B
      |         |
    1 |         | 1
      v    2    v
      C ------> D

from A: dist(C)=1, dist(D)=3 through C, dist(B)=4
```

```
record Step(int vertex, long distance) {}

static long[] dijkstra(List<List<Edge>> graph, int source) {
    long[] distance = new long[graph.size()];
    Arrays.fill(distance, Long.MAX_VALUE);
    distance[source] = 0;

    PriorityQueue<Step> frontier = new PriorityQueue<>(
            Comparator.comparingLong(Step::distance));
    frontier.offer(new Step(source, 0));

    while (!frontier.isEmpty()) {
        Step current = frontier.poll();
        if (current.distance() != distance[current.vertex()]) continue;

        for (Edge edge : graph.get(current.vertex())) {
            if (edge.weight() < 0) {
                throw new IllegalArgumentException("negative edge");
            }
            if (current.distance() > Long.MAX_VALUE - edge.weight()) {
                continue; // or report overflow according to the domain
            }
            long candidate = current.distance() + edge.weight();
            if (candidate < distance[edge.to()]) {
                distance[edge.to()] = candidate;
                frontier.offer(new Step(edge.to(), candidate));
            }
        }
    }
    return distance;
}
```

Java's `PriorityQueue` has no efficient decrease-key operation. Insert the improved immutable entry and discard it later if its distance is stale. Complexity is O((V + E) log V) with this adjacency-list approach. Dijkstra is incorrect with negative edges because a supposedly finalized distance can later improve.

A negative cycle reachable from the source means no finite shortest path for vertices reachable through that cycle. Bellman-Ford detects it by checking for improvement after V - 1 relaxation rounds.

Union-Find for evolving connectivity

Disjoint-set union, or Union-Find, maintains components under edge additions. Each set has a representative. Path compression shortens find paths, and union by rank or size attaches the smaller tree under the larger.

```
before find(4):        after path compression:

4 -> 3 -> 2 -> 1      4 ----\
3 -> 2 -> 1           3 -----+-> 1
2 -> 1                2 ----/
```

```
final class DisjointSet {
    private final int[] parent;
    private final int[] size;

    DisjointSet(int n) {
        parent = new int[n];
        size = new int[n];
        for (int i = 0; i < n; i++) {
            parent[i] = i;
            size[i] = 1;
        }
    }

    int find(int x) {
        while (x != parent[x]) {
            parent[x] = parent[parent[x]];
            x = parent[x];
        }
        return x;
    }

    boolean union(int a, int b) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return false;
        if (size[ra] < size[rb]) { int t = ra; ra = rb; rb = t; }
        parent[rb] = ra;
        size[ra] += size[rb];
        return true;
    }
}
```

The amortized time is effectively constant, formally inverse-Ackermann. Union-Find answers whether vertices are connected after additions; it does not naturally handle edge deletion or produce an actual path.

Minimum spanning trees

For a connected weighted undirected graph, a minimum spanning tree connects every vertex with V - 1 edges and minimum total weight. It minimizes the selected network, not necessarily the path between every pair.

Kruskal sorts edges by weight and uses Union-Find to add an edge only when it joins different components. Time is dominated by sorting: O(E log E). Prim grows one tree through the cheapest crossing edge and commonly uses a priority queue: O(E log V) with adjacency lists.

Do not use a minimum spanning tree when the requirement is shortest routes from a source. Those objectives differ. MST is useful for network layout, clustering, and cost-minimal connectivity; Dijkstra builds a shortest-path tree under nonnegative weights.

Strongly connected components

In a directed graph, vertices are strongly connected when each can reach the other. Contracting every strongly connected component produces a DAG. This is useful for identifying mutually dependent services, recursive modules, or groups that must be handled together.

Kosaraju performs DFS order plus traversal of the reversed graph. Tarjan uses one DFS with discovery indices, low-link values, and an active stack. Both are O(V + E). A senior interview answer should explain the component meaning and condensation DAG even if full Tarjan code is not required.

Graphs in production systems

Service dependency graphs help assess blast radius, but an observed call graph changes over time and may omit rare edges. Build graphs have deterministic prerequisites. Authorization graphs may need temporal validity and deny precedence. Fraud graphs can be too large for one JVM and require partitioned storage, approximate neighborhood queries, or specialized graph databases.

Graph scale changes implementation choices. Object-per-edge designs create heavy allocation and pointer chasing. Compressed sparse row formats use contiguous arrays for static graphs. Large traversals need bounded result size, cancellation, timeouts, and possibly bidirectional search. Distributed traversal is dominated by network movement and partition boundaries rather than textbook operation count.

Many production graphs are temporal. A service dependency may be valid only for one deployment version; an account relationship may have `validFrom` and `validTo`; a route weight may be the current latency estimate rather than a timeless fact. A query such as "what was reachable during the incident?" requires an as-of snapshot, not today's adjacency list. Store version or validity metadata and define whether traversal observes one timestamp, a transaction snapshot, or a weakly consistent live graph.

High-degree vertices deserve special handling. A celebrity, shared database, or platform gateway can dominate traversal cost even when total graph size seems manageable. Degree limits, sampling, precomputed neighborhoods, and per-vertex work budgets may be required. Average degree hides these hubs; monitor degree percentiles and maximum degree.

Snapshot consistency matters. If vertices and edges mutate during traversal, the result may correspond to no coherent graph version. Use immutable snapshots, versioned edges, locks, or explicitly weakly consistent semantics. Avoid invoking remote services while traversing an in-memory dependency graph; first compute the plan, then execute with bounded concurrency and failure handling.

Authorization graphs need fail-closed semantics and explainability. A cached reachability answer must be invalidated when memberships or deny edges change, and a decision should retain the path or rule evidence needed for audit. The fastest traversal is not sufficient if stale positive access can outlive a revocation.

Testing graph algorithms

Test empty graphs, isolated vertices, self-loops, parallel edges, disconnected components, chains, stars, dense graphs, cycles, DAGs with multiple valid orders, zero-weight edges, negative edges, and numeric overflow. Verify properties:

- every emitted path uses real edges;
- BFS distances match a simple oracle on small graphs;
- topological order places every source before its destination;
- component labels agree with mutual reachability for undirected graphs;
- Dijkstra matches Bellman-Ford on small nonnegative graphs;
- an MST has V - 1 edges, no cycle, and spans the component;
- traversal does not mutate the graph unless documented.

Random graph generation catches assumptions about vertex numbering, neighbor order, and disconnected input. Keep deterministic seeds for reproducibility.

Common senior-level traps

- Treating an undirected edge as one adjacency entry.
- Omitting isolated vertices because they never appear in an edge list.

- Marking BFS vertices only when dequeued and creating duplicates.
- Claiming a unique DFS or topological order without ordering rules.
- Using one cycle algorithm for both directed and undirected graphs.
- Running Dijkstra with negative weights.
- Confusing a minimum spanning tree with shortest paths.
- Using recursion on unbounded graph depth.
- Mutating vertex fields used by hash collections.
- Claiming adjacency-list edge lookup is always O(1).
- Ignoring stale priority-queue entries, overflow, snapshots, and result bounds.

A strong interview answer

I begin by defining direction, weight, duplicate-edge, self-loop, and mutation semantics. For sparse graphs I normally use adjacency lists, giving O(V + E) traversal; matrices suit small dense graphs or constant-time edge checks. BFS uses a queue and gives unweighted shortest edge counts. DFS exposes branch and postorder structure for cycles and dependencies. Kahn's algorithm topologically orders a DAG. Dijkstra requires nonnegative weights; Union-Find supports connectivity under additions; Kruskal or Prim finds minimum-cost connectivity rather than source shortest paths. In production I use immutable IDs, bound traversal, consider recursion depth and snapshot consistency, and choose a memory layout appropriate to graph scale.

Chapter summary

Graphs model relationships whose direction, weight, multiplicity, and time semantics define the problem. Representation controls memory and edge-access cost. BFS explores layers; DFS exposes nested structure; topological sorting resolves DAG prerequisites; shortest-path algorithms depend on weight guarantees; Union-Find maintains additive connectivity; and spanning trees minimize network cost. Production correctness also depends on identity, snapshots, bounds, data quality, and operational scale.

Revision Notes

- Define directed/undirected, weighted/unweighted, simple/multigraph, and mutation semantics.
- Adjacency lists use O(V + E) space; matrices use O(V squared).
- BFS uses a queue and finds unweighted shortest edge counts.
- Mark BFS vertices when enqueued to prevent duplicate frontier entries.
- DFS uses recursion or an explicit stack and supports structural analysis.
- Recursive graph depth can overflow the Java stack.
- Run a traversal from every unvisited vertex to find undirected components.
- Undirected cycle detection must ignore the parent edge.
- Directed DFS uses WHITE, GRAY, and BLACK states; an edge to GRAY is a cycle.
- Topological ordering exists only for a DAG.
- Kahn's algorithm uses indegrees; fewer than V emitted vertices means a cycle.
- Dijkstra requires nonnegative weights and a min-priority frontier.
- Stale Dijkstra entries are skipped when they no longer match the best distance.
- Bellman-Ford supports negative edges and detects reachable negative cycles.
- Union-Find supports connectivity under additions with path compression and union by size/rank.
- Kruskal and Prim find minimum spanning trees, not source shortest paths.
- Strongly connected components condense a directed graph into a DAG.
- Production traversal needs immutable identity, bounds, cancellation, and snapshot semantics.

Likely interview question: BFS versus DFS?

BFS explores by distance layers, uses a queue, and finds shortest paths by edge count in an unweighted graph. DFS follows one branch, uses the call stack or an explicit stack, and is natural for cycle, postorder, component, and dependency reasoning. Both are O(V + E) with adjacency lists.

Likely interview question: Why does Dijkstra fail with negative edges?

Dijkstra relies on the smallest frontier distance being final. A later path through a negative edge can reduce a vertex already treated as final, invalidating that proof. Use an algorithm such as Bellman-Ford when reachable negative edges are permitted.

Likely interview question: How do you detect a dependency cycle?

For a directed graph, DFS can detect an edge to a vertex active on the current path, or Kahn's algorithm can reveal that not all vertices reach indegree zero. In production, also return a concrete cycle path so owners can correct the dependency.

Likely interview question: Adjacency list or matrix?

Use a list for sparse graphs and full traversals because memory is O(V + E). Use a matrix when the graph is small and dense or constant-time arbitrary edge checks dominate. The degree distribution, mutation rate, identifiers, and memory layout also matter.